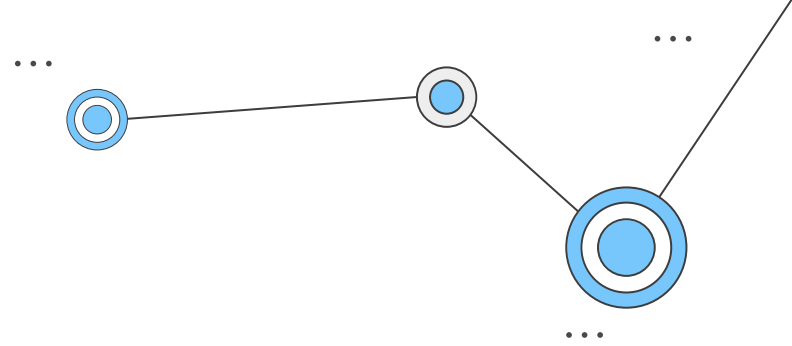




Intodution To **Web Technology**

**Topic: Complex HTTP
Interactions**

Muhammad Abu Zar Tamimi

Table of Contents

01

Complex HTTP Interactions

02

Caching

3

Cookie Coordination:

04

Authorization: challenge and response

05

Re-factoring

06

Requesting supporting data items

07

Multimedia support





01

Complex HTTP Interactions

1- Complex HTTP Interactions

- Now that we have covered the basics of request and response processing, let us move on to situations where the interplay (interaction) of requests and responses yields more sophisticated functionality. The areas we mentioned earlier were
 - caching, authorization, cookies, the request of supporting data items, and the processing of other complex response headers
- Let us examine how the modules in our browser architecture interact to provide this functionality.

02

Caching

2- Caching

- When we speak of *caching*, we are referring to the persistence in some storage mechanisms of generated and retrieved server resources to improve the performance of the response generation process.
- There is **server-side caching**, which relieves(dismisses) the server from regenerating a response and ensured that the stored response is still deemed(supposed) usable (i.e. has not expired).
- And in **client-side caching**, it can relieve the client of the responsibility for re-requesting a response from the server
- This can yield enormous savings in data transmission time.
- Support for browser caching requires three components:
 - Request Generation module,
 - Response Generation module
 - and Caching Support module

2- Caching

- An example of an HTTP response with the defined expiration date is shown below. This entry would expire one day after the original request was made:
 - **HTTP/1.1 200 OK**
 - **Date: Sun, 13 May 2001 12:36:04 GMT**
 - **Content-Type: image/jpeg**
 - **Content-Length: 34567**
 - **Cache-Control: private**
 - **Expires: Mon, 14 May 2001 12:36:04 GMT**
 - **Last-Modified: Sun, 13 May 2001 12:36:04 GMT**
 - ...
- The next time this particular resource is desired, the cached copy may be used, at least until the specified expiration date.



03

Cookie Coordination



3- Cookie Coordination

- HTTP is a stateless protocol, but *cookies can maintain the state during a browser session.*
- ***The principle is simple:*** In responses sent to browsers, servers can include key/value pairs (cookies), which the clients are responsible for remembering.
- Every time the client sends a request back to a server for which it has received a cookie, it must include it in the request.
- This helps servers to *identify specific browser instances, allowing them to associate the requests with particular users.*
- Servers transmit cookies to browsers:

Set-Cookie:

`<name>=<value>[; expires=<date>][; path=<path>]
[; domain=<domain name>][; secure]`

- Browsers send identifying cookies back to appropriate servers by including a set of Key/Value pairs originally sent by the server:

Cookie: `key1=value1; key2=value2; . . .`



04

Authorization



4- Authorization

- The authorization mechanism associated with *basic HTTP authentication* is an ongoing interchange.
- Mechanisms exist on virtually all web servers to 'protect' selected resources from web clients.
- Usually, this is accomplished through a combination of IP address security and some form of *Access Control List (ACL)* describes the identifiers and passwords for users that are allowed to access the resources.
- The Webmaster associates groups of resources (usually directory sub-trees) with specific realms (area of storage).
- Let's start with a Web client request for a protected resource.

GET /protected/index.html HTTP/1.1

Host: secret.resource.com

4- Authorization

- The Web server, however, knows that the resource is protected (and is associated with a particular realm), and sends an appropriate response, with the following **status code**:
HTTP/1.1 401 Not Authorized
Date: Sun, 11 Feb 2001 22:28:31 GMT
WWW-Authenticate: Basic realm ="Top Seekrit"
Content-type: text/html
- This response is the *authentication challenge*. At this point, the client must answer the challenge by providing *authorization credentials*.
- If the credentials provided *do match the user-id and password in the ACL*, the Web server will then finally transmit the contents of the requested resource.



05

Re-Factoring



5- Re-factoring

- There are many similarities between the mechanisms provided for storing cached content, cookie information, and authorization credentials.
- They can all be thought of as a form of *persistence*, since they all represent efforts to store similarly structured information that will be reused later on.
- Therefore browser builds a generalized persistence mechanism for all of these purposes i.e. :
 - In-memory persistence (where persisted information is saved only for the duration of the browser session and never stored permanently),
 - long-term persistence (where persisted information may be placed in some form of permanent data storage so that it lasts beyond the end of the browser session).

5- Re-factoring

	Decision to store	Access key	When to delete	Storage mechanism
Cache	Depends on response headers	URL associated with request	At expiration date or when cache is full	Memory and/or disk
Cookies	Depends on user settings	Domain and path parameters in cookie	At expiration date, or at end of session if no date provided	Memory for cookies that expire at end of session, disk for persisted cookies
Authorization credentials	Always	Server address and authentication realm	At end of session!!	Memory only (never store on disk)



06

Requesting supporting data items



6- Requesting supporting data items

- Even the simplest web page is not 'self-contained'.
- Most pages at the very least, contain references to images found at other URLs, or other mutually linked pages.
- In order to support graphical page rendering properly, a browser must make supplementary requests to retrieve supporting resources.
- The *Content Interpretation module* is responsible for performing this analysis.
- Once it has determined which additional resources are desired, it must tell the *Request Generation module* to *construct and* transmit HTTP requests to obtain these resources.



07

Multimedia Support



7- Multimedia Support

- HTML, plain text, GIF and JPEG are supported “natively” by pretty much all of the modern desktop browsers.
- Apart from images, there are other popular types of data presented via the Web like “Multimedia Objects” (audio and video), and also proprietary formats such as Adobe Acrobat PDF and Macromedia Flash.
- To enable the presentation of these data objects, browsers can provide Plug-ins.
- Plug-ins are program modules closely integrated with the browser, which enable the rendering / presentation of particular kinds of objects within the browser window e.g. <OBJECT> and <EMBED> tag as a generalized approach to embedding multimedia objects.
- OR they can allow the invocation (to tune) of *helper programs* to present these objects e.g.
